16

Short vectors in lattices

In this chapter, we present a polynomial-time algorithm for factoring univariate polynomials with integer coefficients. We will also indicate how the algorithm can be modified so as to also work for bivariate polynomials over a field where we have univariate factorization, such as $\mathbb{Q}$ or a finite field. The main technical ingredient, short vectors in lattices, will be the central topic of this chapter.

16.1. Lattices

The methods we discuss in this chapter deal with computational aspects of the *geometry of numbers*, a mathematical theory initiated by Hermann Minkowski in the 1890s. This theory produces many results about Diophantine approximation, convex bodies, embeddings of algebraic number fields in $\mathbb{C}$, and the ellipsoid method for rational linear programming.

Let $f = (f_1, \ldots, f_n) \in \mathbb{R}^n$. In this chapter, we use the **norm** (or 2-norm, or Euclidean norm) of f, given by

$$\|f\| = \|f\|_2 = \Big(\sum_{1 \le i \le n} f_i^2 \Big)^{1/2} = (f \star f)^{1/2} \in \mathbb{R},$$

where $f \star g = \sum_{1 \le i \le n} f_i g_i \in \mathbb{R}$ is the usual **inner product** of two vectors f and $g = (g_1, \ldots, g_n)$ in $\mathbb{R}^n$ (often written as (f, g), or $\langle f, g \rangle$, or $f \cdot g^T$ in the literature). The vectors f and g are **orthogonal** if $f \star g = 0$.

DEFINITION 16.1. *Let $n \in \mathbb{N}$ and $f_1, \ldots, f_n \in \mathbb{R}^n$ with $f_i = (f_{i1}, \ldots, f_{in})$. Then*

$$L = \sum_{1 \le i \le n} \mathbb{Z} f_i = \{ \sum_{1 \le i \le n} r_i f_i : r_1, \ldots, r_n \in \mathbb{Z} \}$$

*is the **lattice** or $\mathbb{Z}$-module generated by $f_1, \ldots, f_n$. If these vectors are linearly independent, they are a **basis** of L. The **norm of L** is $|L| = |\det(f_{ij})_{1 \le i,j \le n}| \in \mathbb{R}$. Lemma 16.2 below implies that it is well defined, in other words, that the norm is independent of the choice of the generators of L.*

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

LEMMA 16.2. *Let $N \subseteq M \subseteq \mathbb{R}^n$ be lattices, generated by $g_1, \dots, g_n$ and $f_1, \dots, f_n$, respectively, where $f_i = (f_{i1}, \dots, f_{in})$ and $g_i = (g_{i1}, \dots, g_{in})$. Then $\det(f_{ij})_{1 \le i,j \le n}$ divides $\det(g_{ij})_{1 \le i,j \le n}$.*

PROOF. For $1 \le i, j \le n$ there exist $a_{ij} \in \mathbb{Z}$ such that $g_i = \sum_{1 \le j \le n} a_{ij} f_j$. Hence $|\det(g_{ij})| = |\det(a_{ij})| \cdot |\det(f_{ij})|$, and the claim follows. □

If we let $N = M$ in the above lemma, so that $f_1, \dots, f_n$ and $g_1, \dots, g_n$ both generate the same lattice, we see that $|\det(f_{ij})| = |\det(g_{ij})|$. Hence the norm is indeed independent of the choice of basis of L. Geometrically, $|L|$ is the volume of the parallelepiped spanned by $f_1, \dots, f_n$, and Hadamard's inequality (Theorem 16.6) says that $|L| \le \|f_1\| \cdots \|f_n\|$ holds.

EXAMPLE 16.3. We let $n = 2$, $f_1 = (12, 2)$, $f_2 = (13, 4)$ and $L = \mathbb{Z} f_1 + \mathbb{Z} f_2$. Figure 16.1 shows some lattice points of L near the origin of the plane $\mathbb{R}^2$. The norm of L is

$$|L| = \left| \det \begin{pmatrix} 12 & 2 \\ 13 & 4 \end{pmatrix} \right| = 22$$

and equals the area of the blue parallelogram in Figure 16.1. Another basis of L is $g_1 = (1, 2)$ and $g_2 = (11, 0)$, and g_1 is a shortest vector in L with respect to the Euclidean norm $\|\cdot\|$. ◇

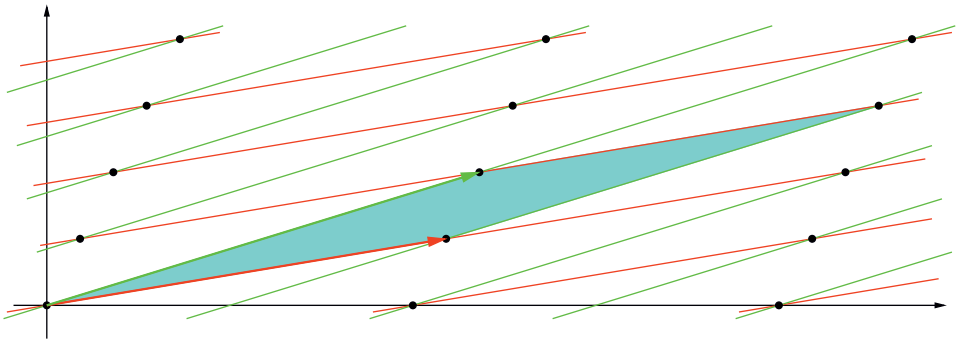


FIGURE 16.1: The lattice in $\mathbb{R}^2$ generated by $(12, 2)$ (red) and $(13, 4)$ (green).

A natural question is to compute a shortest vector in a given lattice. This problem is "$\mathcal{NP}$-hard", and there is no hope for efficient algorithms. But for our current application, the factorization of polynomials with integer coefficients, it will be sufficient to compute a "relatively short" vector, a problem for which Lenstra, Lenstra & Lovász (1982) first gave a polynomial time algorithm. Their "short vector" is guaranteed to be off by not more than a specified factor, which depends on the dimension but not the lattice itself.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

16.2. Lenstra, Lenstra and Lovász' basis reduction algorithm

We briefly review the Gram-Schmidt orthogonalization procedure from linear algebra. Given an arbitrary basis $(f_1,\dots,f_n)$ of $\mathbb{R}^n$, it computes an orthogonal basis $(f_1^*,\dots,f_n^*)$ of $\mathbb{R}^n$ by essentially performing Gaussian elimination on the **Gramian matrix** $(f_i \star f_j)_{1\le i,j\le n} \in \mathbb{R}^{n\times n}$ (Section 25.5). The f_i^* are defined inductively as follows.

$$f_i^* = f_i - \sum_{1\le j<i} \mu_{ij} f_j^*, \text{ where } \mu_{ij} = \frac{f_i \star f_j^*}{f_j^* \star f_j^*} = \frac{f_i \star f_j^*}{\|f_j^*\|^2} \text{ for } 1 \le j < i. \tag{1}$$

In particular, $f_1^* = f_1$. We will call $(f_1^*,\dots,f_n^*)$ the **Gram-Schmidt orthogonal basis** of $(f_1,\dots,f_n)$, and the f_i^* together with the μ_{ij} form the **Gram-Schmidt orthogonalization** (or GSO for short) of $f_1,\dots,f_n$. The GSO has rational coefficients if $f_1,\dots,f_n$ have, and then the cost for computing the GSO is $O(n^3)$ arithmetic operations in $\mathbb{Q}$.

We consider the f_i and f_i^* to be row vectors in $\mathbb{R}^n$, and define three $n\times n$ matrices F, F^*, and M in $\mathbb{R}^{n\times n}$:

$$F = \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix},\ F^* = \begin{pmatrix} f_1^* \\ \vdots \\ f_n^* \end{pmatrix},\ M = (\mu_{ij})_{1\le i,j\le n},$$

where $\mu_{ii} = 1$ for $i \le n$, and $\mu_{ij} = 0$ for $1 \le i < j \le n$. Then M is lower triangular with ones on the diagonal, and (1) reads:

$$F = \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix} = \begin{pmatrix} 1 & & 0 \\ \vdots & \ddots & \\ \mu_{n1} & \cdots & 1 \end{pmatrix} \begin{pmatrix} f_1^* \\ \vdots \\ f_n^* \end{pmatrix} = M \cdot F^*. \tag{2}$$

EXAMPLE 16.4. We let $n = 3$, $f_1 = (1,1,0)$, $f_2 = (1,0,1)$, $f_3 = (0,1,1)$, and calculate $f_1^* = f_1 = (1,1,0)$,

$$\mu_{21} = \frac{f_2 \star f_1^*}{f_1^* \star f_1^*} = \frac{1}{2}, \quad f_2^* = f_2 - \mu_{21} f_1^* = \left(\frac{1}{2}, -\frac{1}{2}, 1\right),$$

$$\mu_{31} = \frac{f_3 \star f_1^*}{f_1^* \star f_1^*} = \frac{1}{2}, \quad \mu_{32} = \frac{f_3 \star f_2^*}{f_2^* \star f_2^*} = \frac{1}{3}, \quad f_3^* = f_3 - \mu_{31} f_1^* - \mu_{32} f_2^* = \left(-\frac{2}{3}, \frac{2}{3}, \frac{2}{3}\right),$$

$$F = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{2} & \frac{1}{3} & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 & 0 \\ \frac{1}{2} & -\frac{1}{2} & 1 \\ -\frac{2}{3} & \frac{2}{3} & \frac{2}{3} \end{pmatrix} = M \cdot F^*.$$

We have $\|f_1\|^2 = \|f_2\|^2 = \|f_3\|^2 = 2$ and $\|f_1^*\|^2 = 2$, $\|f_2^*\|^2 = 3/2$, $\|f_3^*\|^2 = 4/3$. $\diamond$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

The following theorem collects the properties of the Gram-Schmidt orthogonalization that we will need. The proof is left as Exercise 16.2.

THEOREM 16.5.
Let $f_1,\dots,f_n \in \mathbb{R}^n$ be linearly independent, and $f_1^*,\dots,f_n^*$ their Gram-Schmidt orthogonal basis. Let $0 \le k \le n$, and let $U_k = \sum_{1\le i\le k} \mathbb{R} f_i \subseteq \mathbb{R}^n$ be the $\mathbb{R}$-subspace spanned by $f_1,\dots,f_k$.

(i) $\sum_{1\le i\le k} \mathbb{R} f_i^* = U_k$.

(ii) f_k^* *is the projection of* f_k *onto the orthogonal complement*

$$U_{k-1}^{\perp} = \{f \in \mathbb{R}^n : f \star u = 0 \text{ for all } u \in U_{k-1}\}$$

of U_{k-1}, *and hence in particular* $\|f_k^*\| \le \|f_k\|$.

(iii) $f_1^*,\dots,f_n^*$ *are pairwise orthogonal, that is,* $f_i^* \star f_j^* = 0$ *if* $i \ne j$.

(iv) $$\det \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix} = \det \begin{pmatrix} f_1^* \\ \vdots \\ f_n^* \end{pmatrix}.$$

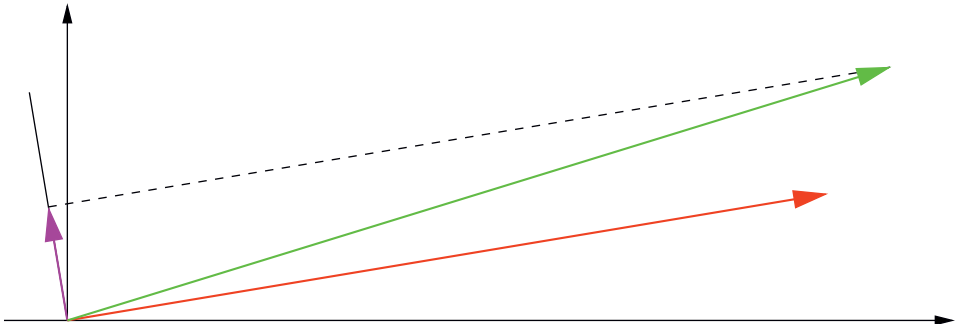


FIGURE 16.2: The Gram-Schmidt orthogonal basis of $(12,2)$ and $(13,4)$.

EXAMPLE 16.3 (continued). We have $f_1^* = f_1 = (12,2)$,

$$\mu_{21} = \frac{f_2 \star f_1^*}{f_1^* \star f_1^*} = \frac{41}{37}, \quad f_2^* = f_2 - \mu_{21} f_1^* = \Big(-\frac{11}{37}, \frac{66}{37}\Big).$$

This is illustrated in Figure 16.2: the vector f_2^* (pink) is the projection of f_2 (green) onto the orthogonal complement of f_1 (red). $\diamond$

An immediate consequence of Theorem 16.5 is the following famous inequality.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

THEOREM 16.6 *Hadamard's inequality.*
Let $A \in \mathbb{R}^{n\times n}$, with row vectors $f_1, \ldots, f_n \in \mathbb{R}^{1\times n}$, and $B \in \mathbb{R}$ such that all entries of A are at most B in absolute value. Then

$$|\det A| \le \|f_1\| \cdots \|f_n\| \le n^{n/2} B^n.$$

PROOF. We may assume that A is nonsingular and the f_i are linearly independent. If $(f_1^*, \ldots, f_n^*)$ is their Gram-Schmidt orthogonal basis, then Theorem 16.5 implies that

$$\left|\det \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix}\right| = \left|\det \begin{pmatrix} f_1^* \\ \vdots \\ f_n^* \end{pmatrix}\right| = \|f_1^*\| \cdots \|f_n^*\| \le \|f_1\| \cdots \|f_n\|.$$

The second inequality follows from noting that $\|f_i\| \le n^{1/2} B$ for all i. □

Of course, the theorem also holds for the column vectors of A.

The next lemma exhibits the connection between the Gram-Schmidt orthogonal basis and short vectors.

LEMMA 16.7. *Let $L \subseteq \mathbb{R}^n$ be a lattice with basis $(f_1, \ldots, f_n)$, and let $(f_1^*, \ldots, f_n^*)$ be its Gram-Schmidt orthogonal basis. Then for any $f \in L \setminus \{0\}$ we have*

$$\|f\| \ge \min\{\|f_1^*\|, \ldots, \|f_n^*\|\}.$$

PROOF. Let $f = \sum_{1\le i\le n} \lambda_i f_i \in L \setminus \{0\}$ be arbitrary, with all $\lambda_i \in \mathbb{Z}$, and let k be the highest index such that $\lambda_k \ne 0$. Substituting $\sum_{1\le j\le i} \mu_{ij} f_j^*$ for f_i yields

$$f = \sum_{1\le i\le k} \lambda_i \sum_{1\le j\le i} \mu_{ij} f_j^* = \lambda_k f_k^* + \sum_{1\le i<k} \nu_i f_i^*$$

for some appropriate $\nu_i \in \mathbb{R}$. Then

$$\begin{aligned} \|f\|^2 = f \star f &= \left(\lambda_k f_k^* + \sum_{1\le i<k} \nu_i f_i^*\right) \star \left(\lambda_k f_k^* + \sum_{1\le i<k} \nu_i f_i^*\right) \\ &= \lambda_k^2 (f_k^* \star f_k^*) + \sum_{1\le i<k} \nu_i^2 (f_i^* \star f_i^*) \ge \lambda_k^2 \cdot \|f_k^*\|^2 \\ &\ge \|f_k^*\|^2 \ge \min\{\|f_1^*\|^2, \ldots, \|f_n^*\|^2\}, \end{aligned}$$

where we used the pairwise orthogonality of the f_i^* and that $\lambda_k \in \mathbb{Z} \setminus \{0\}$. □

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

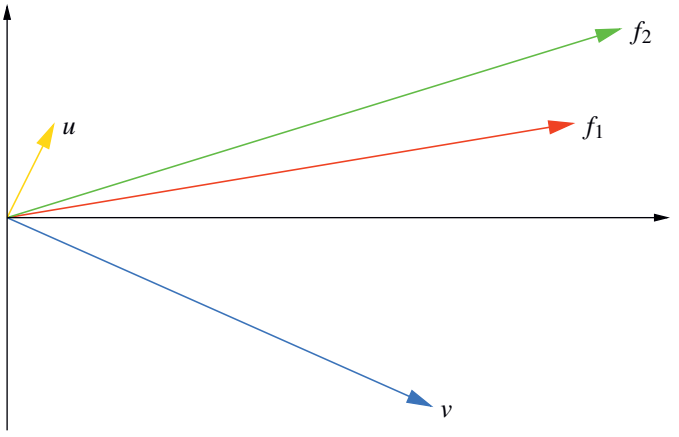


FIGURE 16.3: The vectors computed by the basis reduction algorithm 16.10 for the lattice of Example 16.3.

Our goal is to compute a short vector in L. If the Gram-Schmidt orthogonal basis of $(f_1,\dots,f_n)$ is a basis for the lattice L generated by $f_1,\dots,f_n$, then the lemma says that one of the f_i^* is a shortest vector. But usually the f_i^* are not even in L, as in Example 16.4. Lemma 16.7 provides a lower bound on the length of nonzero vectors in L, in terms of the GSO. With the following definition, we get a similar, though somewhat weaker, bound in terms of the original basis.

DEFINITION 16.8. *Let $f_1,\dots,f_n \in \mathbb{R}^n$ be linearly independent and $(f_1^*,\dots,f_n^*)$ the corresponding Gram-Schmidt orthogonal basis. Then $(f_1,\dots,f_n)$ is* ***reduced*** *if $\|f_i^*\|^2 \le 2\|f_{i+1}^*\|^2$ for $1 \le i < n$.*

THEOREM 16.9.
Let $(f_1,\dots,f_n)$ be a reduced basis of the lattice $L \subseteq \mathbb{R}^n$ and $f \in L \setminus \{0\}$. Then $\|f_1\| \le 2^{(n-1)/2} \cdot \|f\|$.

PROOF. We have $\|f_1\|^2 = \|f_1^*\|^2 \le 2\|f_2^*\|^2 \le 2^2\|f_3^*\|^2 \le \cdots \le 2^{n-1}\|f_n^*\|^2$. Thus $\|f\| \ge \min\{\|f_1^*\|,\dots,\|f_n^*\|\} \ge 2^{-(n-1)/2}\|f_1\|$, using Lemma 16.7. □

We now present an algorithm that computes a reduced basis of a lattice $L \subseteq \mathbb{Z}^n$ from an arbitrary basis. One can use this to find a reduced basis of a lattice in $\mathbb{Q}^n$, by multiplying with a common denominator of the given basis vectors. For $\mu \in \mathbb{R}$, we write $\lceil \mu \rfloor = \lfloor \mu + 1/2 \rfloor$ for the integer nearest to μ.

ALGORITHM 16.10 Basis reduction.
Input: Linearly independent row vectors $f_1,\dots,f_n \in \mathbb{Z}^n$.
Output: A reduced basis $(g_1,\dots,g_n)$ of the lattice $L = \sum_{1\le i\le n} \mathbb{Z} f_i \subseteq \mathbb{Z}^n$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

1. **for** $i = 1, \ldots, n$ **do** $g_i \longleftarrow f_i$
   compute the GSO $G^*, M \in \mathbb{Q}^{n \times n}$, as in (1) and (2), $i \longleftarrow 2$
2. **while** $i \leq n$ **do**
3. **for** $j = i-1, i-2, \ldots, 1$ **do**
4. $g_i \longleftarrow g_i - \lceil \mu_{ij} \rfloor g_j$, update the GSO { replacement step }
5. **if** $i > 1$ and $\|g_{i-1}^*\|^2 > 2\|g_i^*\|^2$
   then exchange g_{i-1} and g_i and update the GSO, $i \longleftarrow i-1$
   else $i \longleftarrow i+1$
6. **return** $g_1, \ldots, g_n$

step	$\begin{pmatrix} g_1 \\ g_2 \end{pmatrix}$	M	$\begin{pmatrix} g_1^* \\ g_2^* \end{pmatrix}$	action
4	$\begin{pmatrix} 12 & 2 \\ 13 & 4 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 \\ \frac{41}{37} & 1 \end{pmatrix}$	$\begin{pmatrix} 12 & 2 \\ -\frac{11}{37} & \frac{66}{37} \end{pmatrix}$	row 2 $\longleftarrow$ row 2 $-$ row 1
5	$\begin{pmatrix} 12 & 2 \\ 1 & 2 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 \\ \frac{4}{37} & 1 \end{pmatrix}$	$\begin{pmatrix} 12 & 2 \\ -\frac{11}{37} & \frac{66}{37} \end{pmatrix}$	exchange rows 1 and 2
4	$\begin{pmatrix} 1 & 2 \\ 12 & 2 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 \\ \frac{16}{5} & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 2 \\ \frac{44}{5} & -\frac{22}{5} \end{pmatrix}$	row 2 $\longleftarrow$ row 2 $-3 \cdot$ row 1
6	$\begin{pmatrix} 1 & 2 \\ 9 & -4 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 \\ \frac{1}{5} & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 2 \\ \frac{44}{5} & -\frac{22}{5} \end{pmatrix}$	

TABLE 16.4: Trace of the basis reduction algorithm 16.10 on the lattice of Example 16.3.

In fact, Algorithm 16.10 does more than required: Lemma 16.12 (iii) below implies that $|\mu_{ij}| \leq 1/2$ holds for the GSO of the reduced basis $(g_1, \ldots, g_n)$. A reduced basis with this additional property is "almost orthogonal".

EXAMPLE 16.3 (continued). Table 16.4 traces the algorithm on the lattice of Example 16.3, and Figure 16.3 depicts the vectors g_i in the computation. We start with $g_1 = f_1 = (12,2)$ (red) and $g_2 = f_2 = (13,4)$ (green). In the second row of Table 16.4, g_2 is replaced by $u = g_2 - \lceil 41/37 \rfloor g_1 = (1,2)$ (yellow). Then $g_1 = f_1$ and $g_2 = u$ are exchanged in the third row. In the last row, $v = g_2 - \lceil 16/5 \rfloor g_1 = f_1 - 3u = (9,-4)$ (blue) is computed, and the algorithm returns the reduced basis $u = (1,2)$ and $v = (9,-4)$. We can see clearly in Figure 16.3 that the final $g_1 = u$ (the yellow vector) is much shorter than the two input vectors f_1, f_2, and that the computed basis u, v (the yellow and the blue vectors) is nearly orthogonal. $\diamond$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

In the example above, the final g_1 is actually a *shortest* vector. This seems to happen quite often, but Theorem 16.9 only guarantees that the norm of the first vector in the computed basis is bigger by a factor of at most $2^{(n-1)/2}$ than the norm of a shortest vector, where n is the dimension of the lattice.

16.3. Cost estimate for basis reduction

THEOREM 16.11.
Algorithm 16.10 correctly computes a reduced basis of L and runs in polynomial time. It uses $O(n^4 \log A)$ arithmetic operations on integers whose length is $O(n \log A)$, where $A = \max_{1 \le i \le n} \|f_i\|$.

The idea of the estimate on the number of arithmetic operations is as follows. Each execution of steps 4 or 5 has polynomial cost, and it is sufficient to bound the number of passes through step 5 with an exchange. In fact, at first glance it is not obvious that the algorithm terminates at all, since the decrease and increase of i in step 5 might continue forever. The crucial point then is to exhibit a value D with the following properties: It is always a positive integer, reasonably small in the beginning, and does not change in the algorithm except that at each exchange step it decreases (at least) by a factor of $3/4$. Therefore only few exchange steps can happen.

To structure the somewhat lengthy proof, we first investigate in the following two lemmas how the GSO of $(g_1, \dots, g_n)$ changes in steps 4 and 5.

LEMMA 16.12. *(i) We consider one execution of step 4, and let $\lambda = \lceil \mu_{ij} \rfloor$ for short. Let G, G^*, M and H, H^*, N in $\mathbb{Q}^{n \times n}$ be the matrices of the g_k, g_k^*, μ_{kl} before and after the replacement, respectively, and $E = (e_{kl}) \in \mathbb{Z}^{n \times n}$ the matrix which has $e_{kk} = 1$ for all k, $e_{ij} = -\lambda$, and $e_{kl} = 0$ otherwise. Then*

$$H = EG, \quad N = EM, \text{ and } H^* = G^*.$$

(ii) The following invariant holds before each execution of step 4:

$$|\mu_{il}| \le \frac{1}{2} \text{ for } j < l < i.$$

(iii) The Gram-Schmidt orthogonal basis $g_1^*, \dots, g_n^*$ does not change in step 4, and after the loop in step 3 we have $|\mu_{il}| \le 1/2$ for $1 \le l < i$.

PROOF. (i) The equality $H = EG$ is just another way of saying that g_i is replaced by $g_i - \lambda g_j$ and all other g_k remain unchanged. Since $j < i$, for any $k \le n$ the space spanned by $g_1, \dots, g_k$ remains the same, and hence the orthogonal vectors $g_1^*, \dots, g_n^*$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

are not changed, which means that $G^* = H^*$. Now the third claim follows from the equations analogous to (2),

$$EMG^* = EG = H = NH^* = NG^*,$$

and the fact that G^* is invertible.

(ii) The invariant is trivial at the beginning of the loop, and we suppose that it holds before step 4. Now the multiplication by E subtracts λ times row j from row i of M, which does not affect the μ_{il} with $j < l$. Finally, μ_{ij} is replaced by $\mu_{ij} - \lambda\mu_{jj}$, which has absolute value at most $1/2$, by the choice of λ, and the invariant holds again before the next execution of step 4.

(iii) is immediate from (i) and (ii). □

$$\begin{pmatrix}
1 & 0 & \cdots & \cdots & \cdots & \cdots & \cdots & 0 \\
\cdot & 1 & \ddots & & & & & \vdots \\
\cdot & \cdot & 1 & \ddots & & & & \vdots \\
\cdot & \cdot & \cdot & 1 & \ddots & & & \vdots \\
\cdot & \cdot & \cdot & \cdot & 1 & \ddots & & \vdots \\
\circ & \circ & \bullet & \cdot & \cdot & 1 & \ddots & \vdots \\
* & * & * & * & * & * & 1 & 0 \\
* & * & * & * & * & * & * & 1
\end{pmatrix}$$

FIGURE 16.5: The effect of one replacement step on the μ_{ij}.

The effect of one replacement step 4 is illustrated in Figure 16.5 for $n = 8$, $i = 6$, and $j = 3$. Each $\cdot$ represents a number which is absolutely less that $1/2$; it is not modified in the current or any later replacement in the loop 3. Each $\circ$ represents a number that is changed in step 4, and the $\bullet$ is μ_{63}, which is reduced to not more than $1/2$ in absolute value by it. (This is also the position of the lonely nonzero off-diagonal entry of E.) The values of $*$ are arbitrary and not changed by the current or any later replacement in the loop 3.

LEMMA 16.13. *Suppose that g_{i-1} and g_i are exchanged in step 5, and denote by h_k and h_k^* the vectors and their Gram-Schmidt orthogonal basis after the exchange, respectively. Then*

(i) $h_k^* = g_k^*$ *for* $k \in \{1, \dots, n\} \setminus \{i-1, i\}$,

(ii) $\|h_{i-1}^*\|^2 < \frac{3}{4}\|g_{i-1}^*\|^2$,

(iii) $\|h_i^*\| \le \|g_{i-1}^*\|$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

PROOF. (i) For such k, we have $g_k = h_k$ and $\sum_{1\le l<k} \mathbb{R} g_l = \sum_{1\le l<k} \mathbb{R} h_l$, and so Theorem 16.5 (ii) implies that $h_k^* = g_k^*$.

(ii) The vector h_{i-1}^* is the component of g_i orthogonal to $\sum_{1\le l<i-1} \mathbb{R} g_l$. Since $g_i = g_i^* + \sum_{1\le l\le i-1} \mu_{i,l} g_l^*$, we have $h_{i-1}^* = g_i^* + \mu_{i,i-1} g_{i-1}^*$ and

$$\|h_{i-1}^*\|^2 = \|g_i^*\|^2 + \mu_{i,i-1}^2 \|g_{i-1}^*\|^2 \le \frac{1}{2}\|g_{i-1}^*\|^2 + \frac{1}{4}\|g_{i-1}^*\|^2 = \frac{3}{4}\|g_{i-1}^*\|^2,$$

by the condition for the exchange, the orthogonality of g_i^* and g_{i-1}^*, and the fact that $|\mu_{i,i-1}| \le 1/2$, by the previous lemma.

(iii) We let $u = \sum_{1\le l<i-1} \mu_{i-1,l} g_l^*$ and $U = \sum_{1\le l<i-1} \mathbb{R} g_l$ for short. Then the vector h_i^* is the component of $g_{i-1} = g_{i-1}^* + u$ orthogonal to $U + \mathbb{R} g_i$. Now Theorem 16.5 implies that $u \in U \subseteq U + \mathbb{R} g_i$. Thus h_i^* is the component of g_{i-1}^* orthogonal to $U + \mathbb{R} g_i$, and hence $\|h_i^*\| \le \|g_{i-1}^*\|$. □

LEMMA 16.14. *At the beginning of each iteration of the loop in step 2, the following invariants hold:*

$$|\mu_{kl}| \le \frac{1}{2} \text{ for } 1 \le l < k < i, \quad \|g_{k-1}^*\|^2 \le 2\|g_k^*\|^2 \text{ for } 1 < k < i.$$

PROOF. The claim is trivial at the beginning of the algorithm. So we assume that the invariants hold at the beginning of step 3 and prove that they hold again at the end of step 5. Lemma 16.12 implies that the first invariant also holds for $k = i$ immediately before step 5, and since an exchange does not affect the μ_{kl} for $k < i-1$, the first invariant holds after step 5 in any case. Again by Lemma 16.12, the g_k^* do not change in steps 3 and 4, and the second invariant is still valid immediately before step 5. Now an exchange in step 5 does not affect the g_k^* for $k \notin \{i-1, i\}$, by Lemma 16.13, and the second invariant holds again after step 5 in any case as well. □

In particular, the above lemma implies that the basis $g_1, \dots, g_n$ is reduced upon termination of the algorithm, and it remains to bound the number of iterations of the loop in step 2. At any stage in the algorithm and for $1 \le k \le n$, we consider the matrix

$$G_k = \begin{pmatrix} g_1 \\ \vdots \\ g_k \end{pmatrix} \in \mathbb{Z}^{k\times n}$$

comprising the first k vectors, their Gramian matrix $G_k \cdot G_k^T = (g_j \star g_l)_{1\le j,l\le k} \in \mathbb{Z}^{k\times k}$, and the **Gramian determinant** $d_k = \det(G_k \cdot G_k^T) \in \mathbb{Z}$. For convenience, we let $d_0 = 1$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

LEMMA 16.15. *For $1 \le k \le n$, we have $d_k = \prod_{1 \le l \le k} \|g_l^*\|^2 > 0$.*

PROOF. Let $1 \le k \le n$, M_k the upper left $k \times k$ submatrix of the transition matrix M, and

$$G_k^* = \begin{pmatrix} g_1^* \\ \vdots \\ g_k^* \end{pmatrix} \in \mathbb{R}^{k \times n}.$$

Then $\det M_k = 1$, $G_k^* \cdot (G_k^*)^T \in \mathbb{R}^{k \times k}$ is a diagonal matrix with diagonal entries $\|g_1^*\|^2, \ldots, \|g_k^*\|^2$, $G_k = M_k G_k^*$, and

$$\begin{aligned} d_k = \det(G_k G_k^T) &= \det(M_k G_k^* (G_k^*)^T (M_k)^T) \\ &= \det(M_k) \cdot \det(G_k^* (G_k^*)^T) \cdot \det(M_k^T) = \prod_{1 \le l \le k} \|g_k^*\|^2. \ \square \end{aligned}$$

LEMMA 16.16. *(i) In steps 3 and 4, none of the d_k changes.*

(ii) If g_{i-1} and g_i are exchanged in step 5 and d_k^* denotes the new value of d_k, for any k, then $d_k^* = d_k$ for $k \ne i-1$ and $d_{i-1}^* \le \frac{3}{4} d_{i-1}$.

PROOF. (i) follows from Lemmas 16.12 and 16.15.

(ii) For $k \ne i-1$ the effect of the exchange on the matrix G_k is a multiplication by a $k \times k$ permutation matrix, whose determinant is 1 or -1. Thus $d_k^* = d_k$. Lemma 16.15 says that $d_{i-1} = \prod_{1 \le l < i} \|g_l^*\|^2$, and Lemma 16.13 yields $d_{i-1}^* \le \frac{3}{4} d_{i-1}$. $\square$

So now we have found our desired loop variant $D = \prod_{1 \le k < n} d_k$, and can bound the number of arithmetic operations. Step 1 takes $O(n^3)$ operations in $\mathbb{Z}$. With the notation as in Lemma 16.12, one execution of step 4 amounts to computing the matrix products EG and EM, at a cost of $O(n)$ operations. Thus the number of operations in $\mathbb{Z}$ used in the loop in step 3 is $O(n^2)$. If an exchange happens in step 5, then only g_{i-1}^*, g_i^*, and rows and columns $i-1$ and i of the transition matrix M change, and they can be updated using $O(n)$ operations, which is dominated by the cost for the loop 3. We always have $1 \le D \in \mathbb{Z}$, and its initial value D_0, at the start of the algorithm, satisfies

$$D_0 = \|f_1^*\|^{2(n-1)} \|f_2^*\|^{2(n-2)} \cdots \|f_{n-1}^*\|^2 \le \|f_1\|^{2(n-1)} \|f_2\|^{2(n-2)} \cdots \|f_{n-1}\|^2 \le A^{n(n-1)},$$

since f_i^* is a projection of f_i for all i. By Lemma 16.16, D does not change in steps 3 and 4, and decreases at least by a factor of $3/4$ if an exchange happens in step 5, so that the number of such exchange steps is bounded by $\log_{4/3} D_0 \in O(n^2 \log A)$. At any stage in the algorithm, let $e \in \mathbb{N}$ denote the number of exchange steps performed so far and e^* the number of times where the else-branch in step 5 has

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

been taken. Since i is decreased by one in an exchange step and increased by one otherwise, the number $i+e-e^*$ is constant throughout the loop of step 2. Initially, it equals 2, and hence $n+1+e-e^*=2$ at termination. Thus the total number of iterations of the loop in step 2 is $e+e^*=2e+n-1\in O(n^2\log A)$, and we get a total of $O(n^4\log A)$ operations in $\mathbb{Z}$, as claimed in Theorem 16.11.

step	$\begin{pmatrix} g_1 \\ g_2 \\ g_3 \end{pmatrix}$	$\begin{pmatrix} \mu_{21} & \\ \mu_{31} & \mu_{32} \end{pmatrix}$	$\begin{pmatrix} \|g_1^*\|^2 \\ \|g_2^*\|^2 \\ \|g_3^*\|^2 \end{pmatrix}$	d_1, d_2 D	action
4	$\begin{pmatrix} 1 & 1 & 1 \\ -1 & 0 & 2 \\ 3 & 5 & 6 \end{pmatrix}$	$\begin{pmatrix} \frac{1}{3} & \\ \frac{14}{3} & \frac{13}{14} \end{pmatrix}$	$\begin{pmatrix} 3 \\ \frac{14}{3} \\ \frac{9}{14} \end{pmatrix}$	3, 14 42	rep(3, 2)
4	$\begin{pmatrix} 1 & 1 & 1 \\ -1 & 0 & 2 \\ 4 & 5 & 4 \end{pmatrix}$	$\begin{pmatrix} \frac{1}{3} & \\ \frac{13}{3} & \frac{-1}{14} \end{pmatrix}$	$\begin{pmatrix} 3 \\ \frac{14}{3} \\ \frac{9}{14} \end{pmatrix}$	3, 14 42	rep(3, 1)
5	$\begin{pmatrix} 1 & 1 & 1 \\ -1 & 0 & 2 \\ 0 & 1 & 0 \end{pmatrix}$	$\begin{pmatrix} \frac{1}{3} & \\ \frac{1}{3} & \frac{-1}{14} \end{pmatrix}$	$\begin{pmatrix} 3 \\ \frac{14}{3} \\ \frac{9}{14} \end{pmatrix}$	3, 14 42	ex(3, 2)
5	$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 2 \end{pmatrix}$	$\begin{pmatrix} \frac{1}{3} & \\ \frac{1}{3} & \frac{-1}{2} \end{pmatrix}$	$\begin{pmatrix} 3 \\ \frac{2}{3} \\ \frac{9}{2} \end{pmatrix}$	3, 2 6	ex(2, 1)
4	$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ -1 & 0 & 2 \end{pmatrix}$	$\begin{pmatrix} 1 & \\ 0 & \frac{1}{2} \end{pmatrix}$	$\begin{pmatrix} 1 \\ 2 \\ \frac{9}{2} \end{pmatrix}$	1, 2 2	rep(2, 1)
6	$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ -1 & 0 & 2 \end{pmatrix}$	$\begin{pmatrix} 0 & \\ 0 & \frac{1}{2} \end{pmatrix}$	$\begin{pmatrix} 1 \\ 2 \\ \frac{9}{2} \end{pmatrix}$	1, 2 2	

TABLE 16.6: Trace of the basis reduction algorithm 16.10 on the lattice $L=\mathbb{Z}(1,1,1)+\mathbb{Z}(-1,0,2)+\mathbb{Z}(3,5,6)$. We have $d_1=\|g_1^*\|^2$, $d_2=\|g_1^*\|^2\|g_2^*\|^2$, $D=d_1d_2$, and $(\det L)^2=d_3=\|g_1^*\|^2\|g_2^*\|^2\|g_3^*\|^2=9$ throughout. Only the relevant values of the μ_{ij} and the squares of the norms of the g_i^* are given, and we have abbreviated a replacement $g_i \longleftarrow g_i-\lceil\mu_{ij}\rfloor g_j$ by $\mathrm{rep}(i,j)$ and an exchange of g_i and g_{i-1} by $\mathrm{ex}(i,i-1)$ in the "action" column.

Table 16.6 traces Algorithm 16.10 and the values of the Gramian determinants d_k and of their product D on a three-dimensional lattice.

We still have the task of bounding the size of (the numerators and denominators of) the rational numbers that occur in the algorithm. The following lemma

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

gives general bounds on the Gram-Schmidt orthogonalization at any stage in the algorithm.

LEMMA 16.17. *Let $g_1, \dots, g_n \in \mathbb{Z}^n$, let G^* and M be its Gram-Schmidt orthogonalization, and $1 \le l < k \le n$. Then*

(i) $d_{k-1} g_k^* \in \mathbb{Z}^n$,

(ii) $d_l \mu_{kl} \in \mathbb{Z}$,

(iii) $|\mu_{kl}| \le d_{l-1}^{1/2} \|g_k\|$.

PROOF. (i) We can write $g_k^* = g_k - \sum_{1 \le l < k} \lambda_{kl} g_l$, with some $\lambda_{kl} \in \mathbb{R}$. (In fact, the λ_{kl} are the coefficients of M^{-1} below the diagonal.) We take the inner product with g_j for some $j < k$. Then $g_k^* \star g_j = 0$, and

$$g_k \star g_j = \sum_{1 \le l < k} \lambda_{kl} (g_l \star g_j).$$

Thus $\lambda_{k1}, \dots, \lambda_{k,k-1}$ form the solution of a $(k-1) \times (k-1)$ system of linear equations. The coefficient matrix is $G_{k-1} \cdot G_{k-1}^T$, its determinant is d_{k-1}, and by Cramer's rule (Theorem 25.6), $d_{k-1} \lambda_{kl} \in \mathbb{Z}$ for each l. Hence $d_{k-1} g_k^* \in \mathbb{Z}^n$.

(ii) $d_l \mu_{kl} = d_l \dfrac{g_k \star g_l^*}{\|g_l^*\|^2} = d_l \dfrac{g_k \star g_l^*}{d_l / d_{l-1}} = d_{l-1}(g_k \star g_l^*) = g_k \star (d_{l-1} g_l^*) \in \mathbb{Z}$, by (i).

(iii) By Lemma 16.15, we have $\|g_l^*\|^2 = d_l / d_{l-1} \ge 1 / d_{l-1}$. Using Cauchy's inequality (Exercise 16.10), we find

$$|\mu_{kl}| = \frac{|g_k \star g_l^*|}{\|g_l^*\|^2} \le \frac{\|g_k\| \|g_l^*\|}{\|g_l^*\|^2} = \frac{\|g_k\|}{\|g_l^*\|} \le d_{l-1}^{1/2} \|g_k\|. \quad \square$$

We have assumed that $\|f_k\| \le A$ for all k. Then A is also an upper bound on the initial Gram-Schmidt orthogonal basis: $\|f_k^*\| \le A$ for all k. Lemmas 16.12 and 16.13 guarantee that the value of $\max\{\|g_k^*\| : 1 \le k \le n\}$ never increases during the algorithm, so that at any stage and for all k, we have

$$\|g_k^*\| \le A \text{ and } d_k = \prod_{1 \le l \le k} \|g_l^*\|^2 \le A^{2k}. \tag{3}$$

LEMMA 16.18. *Let $1 \le k \le n$.*

(i) At any stage in the algorithm, except possibly in steps 3 and 4 when $k = i$, we have $\|g_k\| \le n^{1/2} A$.

(ii) During each execution of step 4, $\|g_i\| \le n(2A)^n$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

PROOF. Initially, we have $\|g_k\| \le A$ for all k. Step 5 does not change the $\|g_k\|$, and it is sufficient to examine what happens in steps 3 and 4. So we assume that the claims are true immediately before step 3. The vectors g_k for $k \ne i$ are not affected by step 4. Let $m_i = \max\{|\mu_{il}| : 1 \le l \le i\}$ be the maximal absolute value in the ith row of M. From $g_i = \sum_{1\le l\le i} \mu_{il} g_l^*$ and the orthogonality of the g_l^*'s, we find

$$\|g_i\|^2 = \sum_{1\le l\le i} \mu_{il}^2 \|g_l^*\|^2 \le n m_i^2 A^2, \text{ and } \|g_i\| \le n^{1/2} m_i A. \tag{4}$$

At the end of loop 3, we have $m_i = 1$, by Lemma 16.12; this concludes the proof of (i).

Lemma 16.17 and (i) imply that at the beginning of loop 3, we have

$$m_i \le \max\{d_{l-1}^{1/2} : 1 \le l < i\} \cdot \|g_i\| \le A^{n-2} \cdot n^{1/2} A = n^{1/2} A^{n-1}. \tag{5}$$

We now consider the replacement in step 4. Since $m_i \ge 1$ and $|\mu_{jl}| \le 1/2$ for $1 \le l < j$, by Lemma 16.14, we have

$$|\mu_{il} - \lceil \mu_{ij} \rfloor \mu_{jl}| \le |\mu_{il}| + |\lceil \mu_{ij} \rfloor| \cdot |\mu_{jl}| \le m_i + (m_i + \frac{1}{2}) \cdot \frac{1}{2} = \frac{3}{2} m_i + \frac{1}{4} \le 2m_i$$

for $1 \le l < j$. For $l = j$, the new value of μ_{ij} is absolutely at most $1/2$, by construction, and also the values of μ_{il} for $l > j$, by Lemma 16.12. Together we find that for each value of j, the value of m_i doubles at most, so that during the loop 3 the value of m_i increases at most by a factor $2^{i-1} \le 2^{n-1}$. Together with (5), this shows that $m_i \le n^{1/2}(2A)^{n-1}$ at all times. Using (4), we have

$$\|g_i\| \le n^{1/2} m_i A \le n(2A)^n. \ \square$$

We now put things together for the final result.

PROOF of Theorem 16.11. We have already shown the correctness, and that the number of arithmetic operations in $\mathbb{Z}$ is $O(n^4 \log A)$. The denominators d_l of the rational numbers computed in the algorithm are at most A^{2n}, and their length is $O(n \log A)$. The numerators are absolutely at most

- $\|g_k\|_\infty \le \|g_k\| \le n(2A)^n$ for g_k, by Lemma 16.18,
- $\|d_{k-1} g_k^*\|_\infty \le \|d_{k-1} g_k^*\| \le A^{2k-2} A \le A^{2n}$ for g_k^*, by Lemma 16.17 and (3),
- $|d_l \mu_{kl}| \le d_l d_{l-1}^{1/2} \|g_l\| \le A^{2l} A^{l-1} n(2A)^n \le n(2A^4)^n$ for μ_{kl}, by Lemmas 16.17 and 16.18,

and hence their length is $O(n \log A)$ as well. $\square$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

COROLLARY 16.19.
Given linearly independent vectors $f_1,\dots,f_n \in \mathbb{Z}^n$ with $\max_{1\le i\le n}\|f_i\| = A$, we can compute a "short" nonzero vector $u \in L = \sum_{1\le i\le n}\mathbb{Z}f_i$ with

$$\|u\| \le 2^{(n-1)/2}\min\{\|f\| : 0 \ne f \in L\}$$

using $O((n^4\log A)\mathsf{M}(n\log A)\log(n\log A))$ or $O^{\sim}(n^5\log^2 A)$ word operations.

PROOF. The claim follows immediately from Theorem 16.11, noting that one arithmetic operation in $\mathbb{Z}$ (addition, multiplication, division with remainder, or gcd) on integers of length m can be performed with $O(\mathsf{M}(m)\log m)$ or $O^{\sim}(m)$ word operations. □

We note that $A \le n^{1/2}B$ when B is an upper bound on the absolute value of the coefficients of the f_i, and hence $n^5\log^2 A \in O^{\sim}(n^5\log^2 B)$, which is indeed polynomial in the input size of about $n^2\log_{2^{64}} B$ words.

16.4. From short vectors to factors

We identify a polynomial $f \in \mathbb{Z}[x]$ of degree n with its coefficient vector in $\mathbb{Z}^{n+1}$, and let $\|f\|$ denote the Euclidean norm of this coefficient vector.

The following lemma states that if two polynomials in $\mathbb{Z}[x]$ have a nonconstant common divisor modulo m for some $m \in \mathbb{N}$ and m is absolutely larger than their resultant, then they have a nonconstant common factor in $\mathbb{Z}[x]$. For a prime m, it is a consequence of Lemma 6.25.

LEMMA 16.20. *Let $f,g \in \mathbb{Z}[x]$ have positive degrees n,k, respectively, and suppose that $u \in \mathbb{Z}[x]$ is nonconstant, monic, and divides both f and g modulo m for some $m \in \mathbb{N}$ with $\|f\|^k\|g\|^n < m$. Then $\gcd(f,g) \in \mathbb{Z}[x]$ is nonconstant.*

PROOF. Suppose that $\gcd(f,g) = 1$ in $\mathbb{Q}[x]$. Then there exist $s,t \in \mathbb{Z}[x]$ such that $sf + tg \equiv \operatorname{res}(f,g) \bmod m$, by Corollary 6.21. Since u divides both f and g modulo m, it divides $\operatorname{res}(f,g)$ modulo m. But u is monic and nonconstant, and thus $\operatorname{res}(f,g) \equiv 0 \bmod m$. Since $|\operatorname{res}(f,g)| \le \|f\|^k\|g\|^n < m$, by Theorem 6.23, it follows that $\operatorname{res}(f,g)$ is zero. This contradiction to our assumption shows that $\gcd(f,g) \in \mathbb{Q}[x]$ is nonconstant. By Corollary 6.10, the gcd of f and g in $\mathbb{Z}[x]$ is also nonconstant. □

The idea of the factoring algorithm is as follows. Suppose that we are given a squarefree primitive polynomial $f \in \mathbb{Z}[x]$ of degree n and have computed a monic polynomial $u \in \mathbb{Z}[x]$ of degree $d < n$ that divides f modulo m for some $m \in \mathbb{N}$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

Then we find a "short" polynomial $g \in \mathbb{Z}[x]$, meaning that $\|g\|^n < m\|f\|^{-\deg g}$, that is also divisible by u modulo m. Then the above lemma gives us a nontrivial factor of f in $\mathbb{Z}[x]$.

To find such a g of degree less than some bound j, we consider the lattice $L \subseteq \mathbb{Z}^j$ generated by (the coefficient vectors of)

$$\{ux^i : 0 \le i < j-d\} \cup \{mx^i : 0 \le i < d\}.$$

An element g of L has the form

$$g = qu + rm \text{ with } q, r \in \mathbb{Z}[x], \quad \deg q < j-d, \quad \deg r < d, \tag{6}$$

and degree less than j. In particular, u divides g modulo m. If, on the other hand, some $g \in \mathbb{Z}[x]$ is of degree less than j and divisible by u modulo m, then we have $g = q^*u + r^*m$ for some $q^*, r^* \in \mathbb{Z}[x]$. Division with remainder by the monic polynomial u yields $q^{**}, r^{**} \in \mathbb{Z}[x]$ with $r^* = q^{**}u + r^{**}$ and $\deg r^{**} < \deg u$. Letting $q = q^* + mq^{**}$ and $r = r^{**}$, we see that g has the form (6), and conclude that

$$g \in L \iff \deg g < j \text{ and } u \text{ divides } g \text{ modulo } m. \tag{7}$$

Thus we can use basis reduction to find a "short" vector $g \in L$ with the desired properties.

EXAMPLE 16.21. We let $f = x^3 - 1 \in \mathbb{Z}[x]$ and $m = 7^6 = 117\,649$. Using factorization over finite fields and Hensel lifting, we find that $f \equiv (x-1)(x-2)(x+3) \bmod 7$ and $f \equiv (x-1)(x-34\,967)(x+34\,968) \bmod 7^6$. Taking $u = x - 34\,967$ and $j = 3$, we consider the lattice $L \subseteq \mathbb{Z}^3$ generated by the coefficient vectors of $ux = x^2 - 34\,967x$, u, and m, namely

$$(1, -34\,967, 0), \quad (0, 1, -34\,967), \quad (0, 0, 117\,649).$$

Since this lattice contains the coefficient vectors of all polynomials in $\mathbb{Z}[x]$ of degree at most two that are divisible by u modulo m, it contains in particular the irreducible factor f_1 of f that is divisible by u modulo m, provided that $\deg f_1 < 3$. By Mignotte's bound 6.33, we then have $\|f_1\| \le 2^{\deg f}\|f\| = 8\sqrt{2}$, and Theorem 16.9 guarantees that the basis reduction algorithm 16.10 will find a polynomial $g \in \mathbb{Z}[x]$ of degree at most two with $\|g\| \le 2^{(3-1)/2}\|f_1\| \le 16\sqrt{2}$. Then

$$\|f\|^{\deg g}\|g\|^{\deg f} \le \sqrt{2}^2(16\sqrt{2})^3 = 16\,384\sqrt{2} < m,$$

and Lemma 16.20 implies that f and g have a common divisor in $\mathbb{Z}[x]$. Indeed, Algorithm 16.10 finds that the three vectors

$$(1, 1, 1), \quad (132, 95, -228), \quad (228, -132, -95)$$

form a reduced basis of L, and if we take $g = x^2 + x + 1$, corresponding to the first vector, then $\gcd(f, g) = g$. $\diamond$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

16.5. A polynomial-time factoring algorithm for $\mathbb{Z}[x]$

Now we are ready to state a polynomial-time algorithm for factoring primitive polynomials in $\mathbb{Z}[x]$. The first four steps are identical to Zassenhaus' algorithm 15.19, with the exception of the values of B in step 1 and of l in step 2, and the factor combination is replaced by the short vector computation.

ALGORITHM 16.22 Polynomial-time factorization in $\mathbb{Z}[x]$.

Input: A squarefree primitive polynomial $f \in \mathbb{Z}[x]$ of degree $n \geq 1$ with $\mathrm{lc}(f) > 0$ and max-norm $\|f\|_\infty = A$.

Output: The irreducible factors $\{f_1, \ldots, f_k\} \subseteq \mathbb{Z}[x]$ of f.

1. **if** $n = 1$ **then return** $\{f\}$
   $b \longleftarrow \mathrm{lc}(f)$, $\quad B \longleftarrow (n+1)^{1/2} 2^n A$
   $C \longleftarrow (n+1)^{2n} A^{2n-1}$, $\quad \gamma \longleftarrow \lceil 2 \log_2 C \rceil$

2. **repeat** choose a prime $p \leq 2\gamma \ln \gamma$, $\quad \overline{f} \longleftarrow f \bmod p$
   until $p \nmid b$ and $\gcd(\overline{f}, \overline{f}') = 1$ in $\mathbb{F}_p[x]$
   $l \longleftarrow \lceil \log_p (2^{n^2/2} B^{2n}) \rceil$

3. { modular factorization }
   compute $h_1, \ldots, h_r \in \mathbb{Z}[x]$ of max-norm at most $p/2$ that are nonconstant, monic, and irreducible modulo p, such that $f \equiv b h_1 \cdots h_r \bmod p$

4. { Hensel lifting }
   call Algorithm 15.17 to compute a factorization $f \equiv b g_1 \cdots g_r \bmod p^l$ with monic polynomials $g_1, \ldots, g_r \in \mathbb{Z}[x]$ of max-norm at most $p^l/2$ such that $g_i \equiv h_i \bmod p$ for $1 \leq i \leq r$

5. { initialize the index set T of modular factors still to be treated, the set G of factors found, and the polynomial f^* still to be factored }
   $T \longleftarrow \{1, \ldots, r\}$, $\quad G \longleftarrow \emptyset$, $\quad f^* \longleftarrow f$

6. **while** $T \neq \emptyset$ **do**

7. choose u among $\{g_t : t \in T\}$ of maximal degree
   $d \longleftarrow \deg u$, $\quad n^* \longleftarrow \deg f^*$
   { find the irreducible factor of f^* that is divisible by u modulo p }
   for $j = d+1, \ldots, n^*$ **do**

8. { short vector computation }
   call Algorithm 16.10 to compute a "short" vector g^* in the lattice $L \subseteq \mathbb{Z}^j$ generated by the coefficient vectors of

$$\{u x^i : 0 \leq i < j - d\} \cup \{p^l x^i : 0 \leq i < d\},$$

   and denote the corresponding polynomial also by g^*

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

9. determine by trial division the set $S \subseteq T$ of indices i for which h_i divides g^* modulo p
compute $h^* \in \mathbb{Z}[x]$ with max-norm at most $p^l/2$ satisfying $h^* \equiv b \prod_{i \in T \setminus S} g_i \bmod p^l$
if $\|\mathrm{pp}(g^*)\|_1 \|\mathrm{pp}(h^*)\|_1 \leq B$ **then**
{ $\mathrm{pp}(g^*)$ is the irreducible factor of f^* that is divisible by u modulo p }
$T \longleftarrow T \setminus S, \quad G \longleftarrow G \cup \{\mathrm{pp}(g^*)\}$,
$f^* \longleftarrow \mathrm{pp}(h^*), \quad b \longleftarrow \mathrm{lc}(f^*)$
break the loop 7 and **goto** 6

10. $T \longleftarrow \emptyset, \quad G \longleftarrow G \cup \{f^*\}$

11. **return** G

To detect whether u in step 7 is already a true factor, we would check whether $u^* \mid bf$ in step 7, where $u^* \equiv bu \bmod p^l$ and $\|u^*\|_\infty \leq p^l/2$, before performing any short vector computations in step 8.

THEOREM 16.23.
The algorithm works correctly, and its expected cost in word operations is

$$O\Big(n^6(n+\log A)\mathsf{M}(n^2(n+\log A))(\log n + \mathrm{loglog} A)\Big) \text{ or } O^\sim(n^{10} + n^8 \log^2 A).$$

PROOF. For the correctness proof, we show that the invariants

$$f^* \equiv b \prod_{i \in T} g_i \bmod p^l, \quad b = \mathrm{lc}(f^*), \quad f = \pm f^* \prod_{g \in G} g, \quad \text{each polynomial in } G \text{ is irreducible} \tag{8}$$

hold each time the algorithm passes through step 6. This is clear initially, and we may assume that they hold before step 7. Let $g \in \mathbb{Z}[x]$ be the irreducible factor of f^* that u divides modulo p. Then (8) and Corollary 15.15 imply that u divides g modulo p^l as well. Conversely, if $v \in \mathbb{Z}[x]$ is a divisor of f which is divisible by u modulo p, then g divides v in $\mathbb{Z}[x]$. As in the proof of Theorem 15.3, the condition in step 9 is satisfied if and only if $\mathrm{pp}(g^*)\,\mathrm{pp}(h^*) = \pm f^*$, and since $\deg g^* < j$ in step 8, we conclude that the condition is false as long as $j \leq \deg g$. In particular, step 10 guarantees that the invariant (8) holds at the end of the algorithm if $\#T = 1$ and $g = f^*$ is irreducible.

Now we assume that $\deg g < n^*$ and let $j = 1 + \deg g$. Then the coefficient vector of g is in L, by (7), and Theorem 16.9 and Mignotte's bound 6.33 imply that $\|g^*\| \leq 2^{(j-1)/2}\|g\| < 2^{n/2}B$. By the choice of l, we have

$$\|g^*\|^{j-1}\|g\|^{\deg g^*} < (2^{n/2}B)^n B^n \leq p^l,$$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

and Lemma 16.20 says that $\gcd(g, g^*)$ is nonconstant in $\mathbb{Z}[x]$. Since g is irreducible and $\deg g^* \leq j-1 = \deg g$, we have $g = \pm \operatorname{pp}(g^*)$.

We let $h = f^*/g$ and $S \subseteq T$ be as in step 9. As in the proof of Theorem 15.20, the uniqueness of Hensel lifting (Theorem 15.14) implies that $\operatorname{lc}(g)h \equiv h^* \bmod p^l$ in step 9. Since $p^l/2$ is larger than the Mignotte bound bB on $\|bh\|_\infty$, we have $\operatorname{lc}(g)h = h^*$, $h = \operatorname{pp}(h^*)$, and $f^* = \pm \operatorname{pp}(g^*)\operatorname{pp}(h^*)$, and Corollary 6.33 implies that the condition in step 9 is true. The actions taken in the **then** clause ensure that the invariants (8) hold at the next pass through step 6. This proves that the algorithm will indeed return the factor g of f.

The cost of the algorithm is dominated by the cost for the short vector computations in step 7. We have $\|v\| \leq j^{1/2}\|v\|_\infty \leq n^{1/2}p^l$ for all generators v of L in step 7. Letting $\delta = \log(n^{1/2}p^l) \in O(n^2 + n\log B) = O(n^2 + n\log A)$, Corollary 16.19 implies that one short vector computation takes $O(j^4\delta \mathsf{M}(j\delta)\log(j\delta))$ word operations. Let $f_1, \ldots, f_k \in \mathbb{Z}[x]$ be the irreducible factors of f. By what we have shown above, the value j in step 7 runs through $j = 2, \ldots, 1 + \deg f_i$ for each irreducible factor f_i. Now $\sum_{1\leq i\leq k}(1+\deg f_i) = k + n \leq 2n$, and hence

$$\begin{aligned}
&\sum_{1\leq i\leq k} \sum_{2\leq j\leq 1+\deg f_i} j^4\delta \mathsf{M}(j\delta)\log(j\delta) \\
&\qquad \leq \sum_{1\leq i\leq k} (1+\deg f_i)^5 \delta \mathsf{M}((1+\deg f_i)\delta)\log((1+\deg f_i)\delta) \\
&\qquad \in O(n^5\delta \mathsf{M}(n\delta)\log(n\delta)),
\end{aligned}$$

by the superlinearity properties of M (Section 8.3). This establishes the time estimate. □

We might also replace steps 1 through 4 of Algorithm 16.22 by the first three steps of the big prime algorithm 15.2, yielding an algorithm with the same asymptotic time bound.

EXAMPLE 16.24. Let $f = 6x^4 + 5x^3 + 15x^2 + 5x + 4 \in \mathbb{Z}[x]$, as in Example 15.4. Choosing $p = 5$ in step 2 of Algorithm 16.22, we would find $l = 39$. Since the coefficients in the algorithm become fairly large with that value of l, we take $l = 6$ for illustration. We skip steps 3 and 4, since we have already found that $f \equiv 6g_1g_2g_3g_4 \bmod 5^6$ in Example 15.4 on page 455, where

$$g_1 = x - 5136, \quad g_2 = x - 984, \quad g_3 = x - 72, \quad g_4 = x - 6828.$$

Thus $r = 4$. In step 7, we choose $u = g_1 = x - 5136$, and then $d = \deg u = 1$ and $n^* = \deg f^* = 4$. We start the loop in step 7 with $j = 2$. Then the lattice in step 8 is generated by the coefficient vectors $(1, -5136)$ and $(0, 15\,625)$ of the two polynomials u and p^l. Algorithm 16.10 finds the reduced basis consisting

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

of the two vectors $(73,72)$ and $(-143,73)$, and the polynomial g^* in step 8 is $g^* = 73x + 72$. In step 9, we find that only $g_1 \bmod 5 = x - 1$ divides $g^* \bmod 5$, whence $S = \{1\}$ and

$$h^* \equiv 6g_2g_3g_4 \equiv 6x^3 - 420x^2 - 840x - 1728 \bmod 5^6.$$

Now both g^* and h^* are primitive, $\|\operatorname{pp}(g^*)\|_1 \|\operatorname{pp}(h^*)\|_1 \geq \|\operatorname{pp}(g^*)\|_\infty \|\operatorname{pp}(h^*)\|_\infty > B \approx 3219.9$, and in fact $f \neq \pm \operatorname{pp}(g^*)\operatorname{pp}(h^*)$, which can also seen by comparing leading coefficients, and we continue the loop 7 with $j = 3$.

So now we consider the lattice generated by the vectors

$$(1,-5136,0), \quad (0,1,-5136), \quad (0,0,15\,625).$$

Exercise 16.6 shows that a "short" vector in this lattice is $(3,1,1) \in \mathbb{Z}^3$, and hence $g^* = 3x^2 + x + 1$ in step 8. Now both $g_1 \bmod 5 = x - 1$ and $g_3 \bmod 5 = x - 2$ divide $g^* \bmod 5$, so that $S = \{1,3\}$, $h^* \equiv 6g_2g_4 = 6x^2 + 3x + 12 \bmod 5^6$, $\operatorname{pp}(g^*) = g^*$, and $\operatorname{pp}(h^*) = 2x^2 + x + 4$ in step 9. In fact, we have $f = \operatorname{pp}(g^*)\operatorname{pp}(h^*)$, g^* is an irreducible factor of f (although we started with a smaller value for l as required), and the assignments in the **if** clause yield $T = \{2,4\}$, $G = \{x - 984, x - 6828\}$, $f^* = 2x^2 + x + 4$, and $b = 2$. The next iteration of the **while** loop 6 would reveal that f^* is irreducible, as we have already seen in Example 15.4. $\diamond$

One might run factor combination and the short vector algorithm concurrently (on one or, even better, on two processors) after Hensel lifting, and take the result from whoever finishes first. This hybrid algorithm is reasonably fast on all inputs, at a cost of at most doubling the overall running time.

COROLLARY 16.25.
A polynomial $f \in \mathbb{Z}[x]$ of degree $n \geq 1$ and with max-norm $\|f\|_\infty = A$ can be completely factored in $\mathbb{Q}[x]$ with an expected number of

$$O\Big(n^6(n + \log A)\mathsf{M}(n^2(n + \log A))(\log n + \operatorname{loglog} A)\Big) \text{ or } O^\sim(n^{10} + n^8 \log^2 A)$$

word operations.

PROOF. We replace the call to Algorithm 15.19 (using factor combination) by a call to Algorithm 16.22 (using basis reduction) in step 4 of the complete factorization algorithm 15.5. The dominant cost is for the calls to Algorithm 16.22 in step 4 for the polynomials $g_1, \ldots, g_s$ in the squarefree decomposition of $\operatorname{pp}(f)$. For $1 \leq i \leq s$, let $n_i = \deg g_i$ and $A_i = \|g_i\|_\infty \leq (n_i + 1)^{1/2} 2^{n_i} A$, using Mignotte's bound 6.33, so that $\log A_i \in O(n_i + \log A)$. Then Theorem 16.23 implies that the cost for the ith call is

$$O(n_i^6(n_i + \log A)\mathsf{M}(n_i^2(n_i + \log A))(\log n_i + \operatorname{loglog} A))$$

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

word operations, and the claim follows from $n_1 + \cdots + n_s \leq n$ and the superlinearity of M (Section 8.3). □

Polynomial-time factoring in $F[x,y]$. Algorithm 16.22 can be adapted to the factorization of polynomials in $F[x,y]$, where F is a field with effective univariate factorization. This is the case, for example, when F is a finite field (Chapter 14), or when $F = \mathbb{Q}$, by the above. The variant of Algorithm 16.22 is then merely a (deterministic) polynomial time **reduction** from bivariate to univariate polynomial factorization.

The main difference is that basis reduction is much easier over $F[y]$ than over $\mathbb{Z}$. The appropriate norm is the **maximum norm** $\|f\|_\infty = \max_{1 \leq i \leq n} \deg f_i$ for a vector $f = (f_1, \ldots, f_n) \in F[y]^n$. In this case, one can always find a *shortest* vector in an n-dimensional lattice in $F[y]^n$ in polynomial time (Exercise 16.12).

16.6. Factoring multivariate polynomials

We have discussed at length the factorization of univariate polynomials over finite fields and over the rational numbers, and briefly indicated a method for bivariate polynomials. What about more variables? This section tells the story of this exciting subject; a precise description of the algorithms and relevant theorems is beyond the scope of this book.

The first issue that distinguishes this problem from the univariate and bivariate cases is that of the appropriate *data structure* or *representation*. We assume we have a polynomial $f \in F[x, \ldots, x_t]$ of total degree n. In the **dense representation**, we write down every term of total degree at most n. For the Fermat polynomial

$$f = x^3 + y^3 - z^3 \in \mathbb{Q}[x,y,z] \tag{9}$$

this reads:

$$\begin{aligned} f \;=\; & 1 \cdot x^3 + 0 \cdot x^2 y + 0 \cdot x^2 z + 0 \cdot x^2 + 0 \cdot xy^2 + 0 \cdot xyz + 0 \cdot xz^2 \\ & + 0 \cdot xy + 0 \cdot xz + 0 \cdot x + 1 \cdot y^3 + 0 \cdot y^2 z + 0 \cdot y^2 \\ & + 0 \cdot yz^2 + 0 \cdot yz + 0 \cdot y + (-1) \cdot z^3 + 0 \cdot z^2 + 0 \cdot z + 0 \cdot 1 \end{aligned} \tag{10}$$

Throughout this book, we have assumed (at least implicitly) this representation for univariate and bivariate polynomials as inputs to algorithms, such as gcd computations or factorization. (In examples, the format like (9) is used.) Multivariate polynomials can be factored in random polynomial time in the length of this dense representation over the usual fields of relevance to computer algebra, such as finite fields, the rational numbers, and finite algebraic and transcendental extensions of these. In fact, it is not hard to adapt the gcd algorithms of Chapter 6 and the bivariate factorization in the previous section to this situation. Is the problem then

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

completely solved? Obviously not, because the drawback of this representation is already visible in the small Example (10) above: it is much too large. It has

$$\alpha_{t,n} = \binom{t+n}{n}$$

terms; in the example, $\alpha_{3,3} = 20$. This number grows exponentially with t and n, and our objects become unmanageable even for reasonable values of t and n. Exercise 16.16 describes a factoring algorithm, based on the **Kronecker substitution**.

It seems clear what to do. The **sparse representation** as in (9) is much more concise and readable than (10). In general, it consists of a list of coefficients and exponents $(a_k, i_{k1}, \ldots, i_{kt})$ so that

$$f = \sum_k a_k \cdot x_1^{i_{k1}} \cdots x_t^{i_{kt}}.$$

If the list consists of s entries, then clearly the length is at least s. But this is not enough; we have to bring the degree into play, since otherwise arbitrarily large degrees might occur. So we consider the length of a list entry $(a_k, i_{k1}, \ldots, i_{kt})$ to be $1 + i_{k1} + \cdots + i_{kt}$; if we count word operations, say over $\mathbb{Q}$, then the summand 1 has to be replaced by the length of a_k. This convention for the length can be expressed by saying that the individual degrees $i_{k1}, \ldots$ are encoded in unary. One might think that the binary encoding for the exponents is more natural. But then the degree may be exponential in the length, and even univariate polynomials become unmanageable. For very simple questions no polynomial-time answer is known in this ultra-concise encoding; for example: given two polynomials in this representation, does the first one divide the second one?

This sparse representation is the natural mathematical notation, and the user of a computer algebra system will want to see her input and output in that format. For a "random" polynomial (with a fixed number of variables and fixed degree), almost all possible coefficients will be nonzero, and there will not be much difference between the sparse and the dense representations. However, natural problems given to a computer algebra system tend to be sparse; see the cyclohexane example in Section 24.4.

Unfortunately, no algorithm for factoring is known that runs in time polynomial in the length of the sparse representation. There are even examples where the output size is more than polynomial in the input size. But even if one allows time polynomial in the combined input plus output size, no direct "sparse" solution is known, but the arithmetic circuit and black box representations discussed below solve the problem.

The key to get over this hurdle is to consider even more concise representations. At first sight, the problem becomes even harder, since the input size (for a fixed polynomial) might be even smaller. But the gain is that the output might also be smaller, and, above all, that new computational methods may be used.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

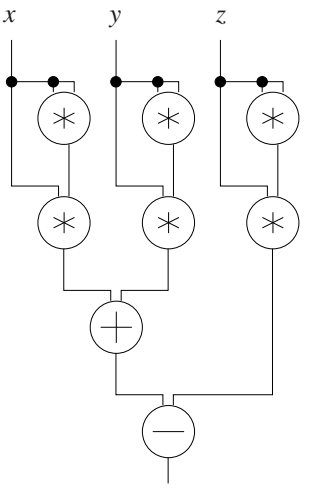


FIGURE 16.7: An arithmetic circuit for $x^3 + y^3 - z^3$.

The first new idea is the **arithmetic circuit representation**, where a polynomial is represented by an arithmetic circuit, as illustrated in Chapters 2 and 8, that computes f using $x_1, \ldots, x_t$ and constants from F as inputs and only addition and multiplication gates. (There is an efficient way to remove division gates if they are also present.) For (9), this looks like Figure 16.7. Equivalently, an arithmetic circuit may be represented by a **straight-line program** such as

$$
\begin{aligned}
g_1 &\longleftarrow x * x \\
g_2 &\longleftarrow g_1 * x \\
g_3 &\longleftarrow y * y \\
g_4 &\longleftarrow g_3 * y \\
g_5 &\longleftarrow z * z \\
g_6 &\longleftarrow g_5 * z \\
g_7 &\longleftarrow g_2 + g_4 \\
g_8 &\longleftarrow g_6 - g_7
\end{aligned}
$$

This approach culminated in Kaltofen's (1989) celebrated result of random polynomial-time factorization in the arithmetic circuit representation. (Actually, the problem of computing a circuit for f from one for f^p in characteristic p is still open in this context.)

The technical key ingredient is an efficient version of *Hilbert's irreducibility theorem*. In Hilbert's (1892) version (see page 586), it says that an irreducible polynomial $f \in \mathbb{Q}[x, y]$ usually yields an irreducible polynomial $f(x, a) \in \mathbb{Q}[x]$ if an integer a is substituted for y. (This is not true for polynomials over finite fields or the complex numbers.) Although there are many generalizations and improve-

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

ments of this in the literature, none of them is strong enough to derive a (probabilistic) polynomial-time algorithm.

However, the situation can be saved by leaving one more variable and considering more general substitutions of the form $ax_1 + bx_2 + c$ for the variables, where a, b, c are chosen randomly from a (sufficiently large) finite subset of the ground field. Then for an irreducible polynomial in $F[x_1, \ldots, x_n]$, the substituted polynomial in $F[x_1, x_2]$ will be irreducible for almost all random choices. An arbitrary polynomial can be mapped to two variables by such a substitution, then the bivariate factoring technology can be applied, and finally Hensel lifting to get back to the original multivariate situation. The role of the efficient Hilbert irreducibility theorem is to insure that one (probably) does not have to worry about irreducible polynomials splitting after the substitution. This phenomenon required factor combination or a short vector computation for substitutions $\mathbb{Z}[x] \longrightarrow \mathbb{Z}_p[x]$ or $F[x, y] \longrightarrow F[x]$, but such methods would not lead to polynomial-time methods in the multivariate case.

An even more powerful technique is the **black box representation**. A polynomial $f \in F[x_1, \ldots, x_n]$ is now given by a "black box" subroutine which on input $a_1, \ldots, a_n \in F$ returns the value $f(a_1, \ldots, a_n) \in F$. We have discussed this type of representation for matrices in Section 12.4. Initially, a polynomial will often be given in some other representation, say the sparse one. It is then easy to build a black box for it. The power of the method is that now these black boxes can be handled efficiently; Kaltofen & Trager (1990) give random polynomial-time algorithms for several problems, including factorization. Finally, the black box representation has to be converted back to human-readable output. There are several interpolation algorithms for achieving this. Now an output polynomial with a few hundred terms is rather useless for the human reader (but possibly useful as input to another procedure). One interpolation method has the beautiful feature that one can tell it to print only about a dozen (or about a hundred) terms (and to say that there are more if that is so).

The black box technology has also been successfully applied to other problems such as the gcd of two multivariate polynomials.

Notes. 16.1. Minkowski (1910) describes the *geometry of numbers* that he invented. Grötschel, Lovász & Schrijver (1993) is a good textbook in this area, and Kannan (1987) present an overview on computational aspects of this theory, including basis reduction and several applications.

Ajtai (1997) shows that computing a shortest vector in a lattice is "$\mathcal{NP}$-hard". This is under probabilistic polynomial-time reductions rather than the usual deterministic reduction for standard $\mathcal{NP}$-hardness. As long as $\mathcal{BPP} \neq \mathcal{NP}$ is considered about as likely as $\mathcal{P} \neq \mathcal{NP}$, this difference does not matter much.

16.2 and 16.3. The Gram-Schmidt orthogonalization procedure is from Schmidt (1907), §3, who states that Gram (1883) has given essentially the same formulas. Hadamard (1893) proved Theorem 16.6. The geometrical idea is that the volume $|\det A|$ of the polytope

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

spanned by $f_1, \ldots, f_n \in \mathbb{R}^n$ is maximal when these vectors are mutually orthogonal; in this case it is precisely equal to $\|f_1\| \cdots \|f_n\|$.

Basis reduction (sometimes called the LLL algorithm) was introduced in the important paper of Lenstra, Lenstra & Lovász (1982), who used it to factor integer polynomials in polynomial time. Their definition of a reduced basis is slightly different from ours. We are grateful to Victor Shoup for his permission to use unpublished lecture notes of his.

Odlyzko (1990) notes that basis reduction *turns out to be fast in practice, and usually finds a reduced basis in which the first vector is much shorter than is guaranteed* [by Theorem 16.9]. *(In low dimensions, it has been observed empirically that this algorithm usually finds the shortest nonzero vector in a lattice.)*

Improvements both on the algorithm and its analysis are from Kaltofen (1983), Schönhage (1984), whose algorithm takes only $O^{\sim}(n^4 \log^2 A)$ word operations for a slightly different notion of reduced basis, Schnorr (1987, 1988), who proves a bound of $O^{\sim}(n^5 \log A + n^4 \log^2 A)$ word operations for his algorithm and approximation quality $(1+\varepsilon)^n \|f\|$ for any $\varepsilon > 0$ in Theorem 16.9 by employing floating point arithmetic, Schnorr & Euchner (1991), who also give efficient implementations, and Storjohann (1996), who achieves $O^{\sim}(n^{3.381} \log^2 A)$ word operations with a modular approach and by employing fast matrix multiplication. The current record is by Koy & Schnorr (2001a) whose *segment reduction* uses $O(n^3 \log n)$ arithmetic operations on n-dimensional lattices with basis vectors of Euclidean norm at most 2^n. Koy & Schnorr (2001b) give a floating point implementation which can handle dimensions of 1000 or more. De Weger (1989) and Storjohann (1996) give fraction-free variants of Algorithm 16.10. Gauß (1801), article 171, describes basis reduction in two dimensions.

16.5. For a unified discussion of the factoring algorithms over $\mathbb{Z}$ and $F[y]$, we refer to von zur Gathen (1984a). By performing a binary search on the degree of f^* instead of the linear search in step 7 of Algorithm 16.22, Lenstra, Lenstra & Lovász (1982) achieve a time bound of $O^{\sim}(n^9 + n^7 \log^2 A)$ word operations.

Schönhage (1984) gives an algorithm with only $O^{\sim}(n^6 + n^4 \log^2 A)$ word operations. His approach is numerical: modular factorization and Hensel lifting are replaced by the computation of a complex root of f to a sufficiently high precision, and Diophantine approximation based on basis reduction (see Section 17.3) is used to find the irreducible factor of f with that root.

Van Hoeij (2002) applies lattice reduction to the factor combination stage of Zassenhaus' algorithm 15.19, and reports: *The result is a practical algorithm that can factor polynomials that are far out of reach for any previous algorithm.* His method is implemented in Shoup's NTL since version 5.2, which takes, e.g., about 5 seconds for the factorization of the seventh Swinnerton-Dyer polynomial, of degree 128, on an 800 MHz Pentium III processor, while the same computation with version 5.1 does not terminate within two weeks. See Hart, van Hoeij & Novocin (2011) for references and practical results.

16.6. The polynomial-time solution for dense multivariate factorization was given by Kaltofen (1985a). An algorithm for sparse factorization is in von zur Gathen & Kaltofen (1985); that paper also has the example with very large factors. Open questions such as the divisibility problem (for exponentially large degrees), are discussed in von zur Gathen (1991a). For the circuit representation, the solution is again by Kaltofen (1989); von zur Gathen (1985) has an irreducibility test in the same data structure. The latter paper introduced this representation into the factorization business, but it has its origins in Strassen (1972, 1973a, 1973b); see also Heintz & Sieveking (1981).

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

The generic determinant polynomial for $n \times n$ matrices is an example where the length of the sparse representation, which is essentially $n!$, is exponential in the length of the arithmetic circuit representation, since there is an arithmetic circuit (with divisions) of size $O(n^3)$ performing Gaussian elimination.

Lang (1983), chapter 9, describes Hilbert's irreducibility theorem and the theory of *Hilbertian fields* for which this theorem holds. Results on specific substitutions that conserve irreducibility are in Sprindžuk (1981, 1983) and Dèbes (1996). Efficient probabilistic versions of Hilbert irreducibility, valid over any field but reducing only to two variables, can be found in Kaltofen (1985b), von zur Gathen (1985), and Kaltofen (1995a). Huang & Wong (1998) give a similar result for more general polynomial ideals.

The important paper of Kaltofen & Trager (1990) introduced the black box method and gave several algorithms discussed above. A seminal idea for sparse interpolation is due to Zippel (1979); several other papers deal with various aspects of interpolation: Ben-Or & Tiwari (1988), Kaltofen & Lakshman (1988), Borodin & Tiwari (1990), Grigoriev, Karpinski & Singer (1990), Clausen, Dress, Grabmeier & Karpinski (1991), Grigoriev, Karpinski & Singer (1994). Freeman, Imirzian, Kaltofen & Lakshman (1988) and Díaz & Kaltofen (1998) describe implementations of the straight-line and the black box technologies, respectively.

Exercises.

16.1 Let $F \in \mathbb{R}^{n\times n}$ be nonsingular. Show that the GSO M, F^* of F is uniquely determined by the conditions that $F = MF^*$, M be lower triangular with ones on the diagonal, and $F^*(F^*)^T$ be diagonal.

16.2* Prove Theorem 16.5.

16.3 We define an inner product $\star$ on the vector space V of continuous real-valued functions on the real interval $[-1,1]$ by $f \star g = \int_{-1}^{1} f(y)g(y)\sqrt{1-y^2}dy$.

(i) Convince yourself that $\star$ is in fact an inner product.

(ii) Compute the Gram-Schmidt orthogonal basis of f_0, f_1, f_2, f_3, where $f_i(x) = x^i$ for $-1 \le x \le 1$. (The resulting polynomials are the monic associates of the first four **Chebyshev polynomials of the second kind**.).

16.4* Let $g_1, \ldots, g_n \in \mathbb{R}^n$ be linearly independent and $L = \sum_{1\le i\le n} \mathbb{Z}g_i$ the lattice that they generate. Prove that for each vector $x \in \mathbb{R}^n$ there is a vector $g \in L$ such that

$$\|x-g\|^2 \le \frac{1}{4}(\|g_1\|^2 + \cdots + \|g_n\|^2).$$

Hint: Induction on n. For the induction step, determine a suitable $\lambda \in \mathbb{Z}$ such that the vector $x - \lambda g_n$ has minimal distance to the hyperplane spanned by $g_1, \ldots, g_{n-1}$.

16.5 (i) Compute the GSO of $(22,11,5)$, $(13,6,3)$, $(-5,-2,-1) \in \mathbb{R}^3$.

(ii) Trace Algorithm 16.10 on computing a reduced basis of the lattice in $\mathbb{Z}^3$ spanned by the vectors from (i). Trace also the values of the d_i and of D, and compare the number of exchange steps to the theoretical upper bound from Section 16.3.

16.6$^{\longrightarrow}$ Compute a "short" vector in the lattice in $\mathbb{Z}^3$ spanned by $(1,-5136,0)$, $(0,1,-5136)$, and $(0,0,15625)$.

16.7* The following algorithm takes an arbitrary nonsingular matrix $A \in \mathbb{Z}^{n\times n}$ and computes a **Hermite normal form** H of A (Notes 4.5), such that $H = UA$ for a matrix $U \in \mathbb{Z}^{n\times n}$ which is unimodular, so that $\det U = \pm 1$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

ALGORITHM 16.26 Hermite normal form.
Input: A matrix $A \in \mathbb{Z}^{n\times n}$ with $\det A \neq 0$.
Output: A matrix $H \in \mathbb{Z}^{n\times n}$ in Hermite normal form such that $H = UA$ for a unimodular matrix $U \in \mathbb{Z}^{n\times n}$.

1. $H \longleftarrow A$
   for $m = n, \ldots, 1$ **do**
2. choose a row index k with $1 \leq k \leq m$ such that
   $|h_{km}| = \min\{|h_{im}| : 1 \leq i \leq m \text{ and } h_{im} \neq 0\}$
   exchange rows k and m of H
3. **if** $h_{mm} \mid h_{lm}$ for $1 \leq l \leq m$ **then goto** 5
4. choose a row index l with $1 \leq l < m$ and $h_{mm} \nmid h_{lm}$
   compute $q \in \mathbb{Z}$ with $|h_{lm} - qh_{mm}| \leq |h_{mm}|/2$ by division with remainder
   subtract q times row m from row l of H
   goto 2
5. **if** $h_{mm} < 0$ **then** multiply row m of H by -1
   for $l = 1, \ldots, m-1, m+1, \ldots, n$ **do**
6. compute $q \in \mathbb{Z}$ with $0 \leq h_{lm} - qh_{mm} < h_{mm}$ by division with remainder
   subtract q times row m from row l in H
7. **return** H

(i) Trace the algorithm on input $A = \begin{pmatrix} 5 & 2 & -4 & 7 \\ 3 & 6 & 0 & -3 \\ 1 & 2 & -2 & 4 \\ 7 & 1 & 5 & 6 \end{pmatrix} \in \mathbb{Z}^{n\times n}$.

(ii) Prove the following invariant of the loop in step 1: $H = UA$ for some unimodular matrix U, and for each column j with $m < j \leq n$, the diagonal entry h_{jj} is positive, the entries above the diagonal are zero, and the entries below the diagonal are nonnegative and less than h_{jj}. Conclude from the invariant and from the fact that $\deg A \neq 0$ that the minimum in step 2 always exists.

(iii) Show that the condition in step 3 is true after at most $\log_2 d$ executions of step 4, where $d = \min\{|h_{im}| : 1 \leq i \leq m \text{ and } h_{im} \neq 0\}$ at the beginning of the current iteration of the **for** loop in step 1. Conclude that the algorithm terminates.

(iv) Let $f_1, \ldots, f_n \in \mathbb{Q}^n$ be linearly independent, $L = \sum_{1\leq i\leq n} \mathbb{Z} f_i$ the lattice that they generate, and $g_1, \ldots, g_n \in L$ be linearly independent as well. Then $M = \sum_{1\leq i\leq n} \mathbb{Z} g_i \subseteq L$ is a **sublattice** of L. Prove that there exists a basis $c_1, \ldots, c_n$ of M which has the form

$$\begin{aligned} c_1 &= h_{11} f_1, \\ c_2 &= h_{21} f_1 + h_{22} f_2, \\ &\vdots \\ c_n &= h_{n1} f_1 + h_{n2} f_2 + \cdots + h_{nn} f_n, \end{aligned} \tag{11}$$

with $h_{ij} \in \mathbb{Z}$ and $h_{ii} \neq 0$ for $1 \leq j \leq i \leq n$.

16.8 Modify the basis reduction algorithm 16.10 so as to work for arbitrary inputs $f_1, \ldots, f_m \in \mathbb{Z}^n$, not necessarily linearly independent, and with arbitrary $m \in \mathbb{N}$.

16.9 With notation as in Lemma 16.13, prove the formula

$$h_i^* = \frac{\|g_i^*\|^2 g_{i-1}^* - \mu_{i,i-1}\|g_{i-1}^*\|^2 g_i^*}{\|h_{i-1}^*\|^2}.$$

Hint: h_i^* is the projection of h_i onto $(\mathbb{R}h_1 + \ldots + \mathbb{R}h_{i-1})^\perp$.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

16.10 Let $x, y \in \mathbb{C}^n$.

(i) Prove the **Cauchy-Schwarz inequality** $|x \star y| \le \|x\|_2 \|y\|_2$. Hint: Consider the inner product $(\|y\|_2 x + \|x\|_2 y) \star (\|y\|_2 x + \|x\|_2 y)$.

(ii) Use (i) to prove the triangle inequality $\|x+y\|_2 \le \|x\|_2 + \|y\|_2$.

16.11* Lemma 16.17 shows that at any stage in the basis reduction algorithm 16.10, $d_l \mu_{kl}$ and $d_{k-1} g_k^*$ have integral coefficients for $1 \le l < k \le n$. By multiplying the entries of the GSO by (and, where possible, dividing out) appropriate d_l's, convert Algorithm 16.10 into a fraction-free algorithm, so that all intermediate coefficients are in $\mathbb{Z}$.

16.12** This exercises discusses basis reduction for polynomials. Let F be a field, $R = F[y]$, and $n \in \mathbb{N}_{>0}$. The **max-norm** of a vector $f = (f_1, \ldots, f_n) \in R^n$ is $\|f\| = \|f\|_\infty = \max\{\deg f_i : 1 \le i \le n\}$. For vectors $f_1, \ldots, f_m \in R$ which are linearly independent over $F(y)$, the field of fractions of R, the ***R*-module** spanned by $f_1, \ldots, f_m$ is $M = \sum_{1 \le i \le m} R f_i$, and $(f_1, \ldots, f_m)$ is a **basis** of M.

(i) Let $f_1, \ldots, f_m \in R^n$ be linearly independent (over $F(y)$), with $f_i = (f_{i1}, \ldots, f_{in})$ for $1 \le i \le m$. We say that the sequence $(f_1, \ldots, f_m)$ is **reduced** if

- $\|f_1\| \le \|f_2\| \le \cdots \le \|f_m\|$, and
- $\deg f_{ij} \le \deg f_{ii}$ for $1 \le j \le n$, with strict inequality if $j < i$, for $1 \le i \le m$.

In particular, we have $\|f_i\| = \deg f_{ii}$ for $1 \le i \le m$. Prove that f_1 is a **shortest vector** in the R-module $M = \sum_{1 \le i \le m} R f_i$, so that $\|f_1\| \le \|f\|$ for all nonzero $f \in M$.

(ii) Consider the following algorithm, from von zur Gathen (1984a).

ALGORITHM 16.27 Basis reduction for polynomials.

Input: Linearly independent (over $F(y)$) row vectors $f_1, \ldots, f_m \in R^n$, where $R = F[y]$ for a field F, with $\|f_i\| < d$ for $1 \le i \le m$.

Output: Row vectors $g_1, \ldots, g_m \in R^n$ and a permutation matrix $A \in R^{n \times n}$ such that $(g_1, \ldots, g_m)$ is a reduced sequence and $(g_1 A, \ldots, g_m A)$ is a basis of $M = \sum_{1 \le i \le m} R f_i$.

1. let $g_1, \ldots, g_m$ be such that $\{g_1, \ldots, g_m\} = \{f_1, \ldots, f_m\}$ and $\|g_i\| \le \|g_{i+1}\|$ for $1 \le i < m$
   $A \longleftarrow \mathrm{id}, \quad k \longleftarrow 1$
2. **while** $k \le m$ **do**
3. { $(g_1, \ldots, g_{k-1})$ is reduced and $\|g_i\| \le \|g_{i+1}\|$ for $1 \le i < m$ }
   $u \longleftarrow \|g_k\|$
4. **for** $i = 1, \ldots, k-1$ **do**
5. $q \longleftarrow g_{ki}$ quo $g_{ii}, \quad g_k \longleftarrow g_k - q g_i$
6. **if** $\|g_k\| < u$ **then**
   $r \longleftarrow \min\{i : i = k \text{ or } (1 \le i < k \text{ and } \|g_i\| > \|g_k\|)\}$
   replace $g_r, \ldots, g_{k-1}, g_k$ by $g_k, g_r, \ldots, g_{k-1}$
   $k \longleftarrow r$, **goto** 2
7. $l \longleftarrow \min\{k \le j \le n : \deg g_{kl} = u\}$
   let $B \in R^{n \times n}$ be the permutation matrix for the exchange of columns k and l
   for $i = 1, \ldots, m$ **do** $g_i \longleftarrow g_i B$
   $A \longleftarrow BA, \quad k \longleftarrow k+1$
8. **return** $g_1, \ldots, g_m$ and A

Show that $M = \sum_{1 \le i \le m} R \cdot g_i A$ holds throughout the algorithm, and conclude that the g_i are always nonzero vectors.

(iii) Assume that the invariants in curly braces are true in step 3. Convince yourself that $\|g_{k-1}\| \le u$ holds during steps 4 and 5 if $k \ge 2$. Show that $g_{ii} \ne 0$ holds in step 5 if $k \ge 2$, so that the division with remainder can be executed, and prove the invariants $\|g_k\| \le u$ and $\deg g_{kj} < u$ for $1 \le j < i$ of the loop 4.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

(iv) Show that $(g_1, \ldots, g_{k-1})$ is reduced and $\|g_i\| \leq \|g_{i+1}\|$ for $1 \leq i < m$ holds each time the algorithm passes through step 3. Conclude that it works correctly if it halts in step 8.

(v) Show that $\|g_i\| < d$ for $1 \leq i \leq m$ holds throughout the algorithm. Prove that the cost for one execution of steps 3 through 7 is $O(nm)$ arithmetic operations (additions, multiplications, and divisions with remainder) in R or $O(nm\mathsf{M}(d))$ operations in F.

(vi) Show that the function $s(g_1, \ldots, g_m) = \sum_{1 \leq i \leq m} \|g_i\|$ never increases in the algorithm and strictly decreases if the condition in step 6 is true. Conclude that the number of times when the latter happens is at most md and that the number of iterations of the loop 2 is at most $(m-1)(md+1)$.

(vii) Putting everything together, show that the running time of the algorithm is $O(nm^3 d\,\mathsf{M}(d))$ or $O^{\sim}(nm^3 d^2)$ arithmetic operations in F.

(viii) Trace the algorithm on the $\mathbb{F}_{97}[y]$-module generated by

$$\begin{gathered}\left(5y^3+44y^2+37y+91,\ 8y^3+86y^2+91y+89,\ 16y^3+65y^2+20y+76\right),\\ \left(8y^3+70y+37,\ 16y^3+7y^2+54y+38,\ 32y^3+23y^2+80y+77\right),\\ \left(16y^2+84y+63,\ 32y^2+15y+19,\ 64y^2+48y+51\right) \in \mathbb{F}_{97}[y]^3.\end{gathered}$$

Mulders & Storjohann (2000) give an algorithm for computing a reduced basis taking only $O(nm^2d^2)$ arithmetic operations in F.

16.13 State and prove the analog of Lemma 16.20 for polynomials in $F[x,y]$ for a field F when $\|\cdot\|_2$ is replaced by $\|f\|_\infty = \deg_y f$.

16.14* Use Exercises 16.12 and 16.13 to adapt Algorithm 16.22 to bivariate polynomials over a field. Prove that your algorithm works correctly and analyze its running time. You may assume that F has effective univariate factorization and is "large enough", so that the modulus p may be chosen linear.

16.15 Let F be a field and $n \in \mathbb{N}$. What is the size of the sparse representation of the polynomial $\prod_{0 \leq i < n}(x + y^{2^i}) \in F[x,y]$? Find an arithmetic circuit representation of size $3n-2$.

16.16* You are to design an algorithm for factoring multivariate polynomials over a field F with efficient univariate factorization. Suppose that $f \in F[x_1, \ldots, x_t]$ has degree less than n in each variable, and consider the **Kronecker substitution** $\sigma: F[x_1, \ldots, x_t] \longrightarrow F[x]$ which maps x_i to $x^{n^{i-1}}$ for $1 \leq i \leq t$. This is a ring homomorphism.

(i) Show that polynomials with degree less than n in each variable can be uniquely recovered from their image under σ. More precisely, let $U \subseteq F[x_1, \ldots, x_t]$ be the vector space of all these polynomials, and $V = \{h \in F[x] : \deg h < n^t\}$. Show that σ gives a vector space isomorphism between U and V.

(ii) Prove that the following procedure correctly factors f: Factor $\sigma(f)$ into irreducible factors $h_1, \ldots, h_r \in F[x]$, and test for each factor combination h of them whether its inverse $\sigma^{-1}(h)$, in the sense of (i), divides f.

(iii) Analyze the cost of your algorithm from (ii). You will first have to estimate the cost for multivariate multiplication (Exercise 8.38). Ignore the time for univariate factorization in this analysis.

(iv) Trace your algorithm on the example $f = -x^4y + x^3z + xz^2 + yz^2 \in \mathbb{F}_3[x,y,z]$.

Research problem.

16.17 Can one compute the gcd of two multivariate polynomials in random polynomial time in the length of the sparse representation plus the degree? Is the output length always polynomial in the input length? As mentioned in Section 16.6, one can factor polynomials in random polynomial time both in the arithmetic circuit and in the black box representation.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press

Il faut bien distinguer entre la géométrie utile et la géométrie curieuse. L'utile est le compas de proportion inventé par Galilée [. . .] Presque tous les autres problèmes peuvent éclairer l'esprit et le fortifier; bien peu seront d'une utilité sensible au genre humain.[1]

Voltaire (1771)

Partager une nuit entre une jolie femme et un beau ciel, le jour à rapprocher ses observations et les calculs, me paraît être le bonheur sur la terre.[2]

Napoléon I. (1812)

But yet one commoditie moare [. . .] I can not omitte. That is the filyng, sharpenyng, and quickenyng of the witte, that by practice of *Arithmetike* doeth insue. It teacheth menne and accustometh them, so certainly to remember thynges paste: So circumspectly to consider thynges presente: And so prouidently to forsee thynges that followe: that it maie truelie bee called the *File of witte.*

Robert Recorde (1557)

[1] One has to distinguish carefully between practical geometry and theoretical geometry. Practical is the rule of proportions invented by Galileo [. . .] Almost all other problems can enlighten the mind and strengthen it; rather few will be of any reasonable usefulness to mankind.

[2] To share a night between a beautiful woman and a clear sky, the day in seeking agreement between one's observations and calculations, seems to me happiness on earth.

https://doi.org/10.1017/CBO9781139856065.020 Published online by Cambridge University Press